Lattice-Based Cryptography and Module-LWE: Foundations, Hardness, and Future Standards

Nikolas Lukowicz
Munich, Germany
niko.lukowicz@tum.de

***Abstract*—The emergence of quantum computers poses a significant threat to classical cryptographic schemes, such as RSA and ECC, which rely on the exponential complexity of mathematical problems like prime factorization and discrete logarithms. These problems, once considered secure, can be efficiently solved by quantum algorithms like Shor's algorithm, rendering classical cryptography insecure in the near future. However, alternative cryptographic methods exist that are resistant to quantum attacks. Among these, lattice-based cryptography stands out as a leading candidate for post-quantum cryptographic security due to its robustness against quantum algorithms, for which no efficient quantum solutions are currently known.**

In this paper, I will explore the foundations of lattice-based cryptography, with a particular focus on the Learning With Errors (LWE) problem. I will also examine the inherent hardness of lattice-based problems used in cryptography, their practical implementations, and their advantages in the context of post-quantum cryptography.

Additionally, I will analyze Module-LWE Key Encapsulation Mechanism (ML-KEM), a specific lattice-based scheme, which was officially selected as a cryptographic standard by NIST in August 2024. This analysis will break down the structure of ML-KEM and highlight its role in ensuring secure communication in a future where quantum computers are capable of breaking traditional cryptosystems.

I. Introduction

Cryptography plays a crucial role in securing information and communications in the digital age. As society becomes increasingly reliant on technology, the need to protect data from unauthorized access, manipulation, and theft has grown exponentially.

Most classical cryptographic schemes, such as RSA and Diffie-Hellman, rely on the hardness of mathematical problems like prime factorization and discrete logarithms [23]. These problems have proven difficult to solve using classical computers due to their exponential complexity, which is why they have formed the backbone of modern cryptographic security for decades. However, the advent of quantum computing poses a significant threat to this foundation. Quantum algorithms, most notably Shor's algorithm, can efficiently solve problems like prime factorization and discrete logarithms, potentially breaking widely used cryptographic systems [26]. Although quantum computers are not yet powerful enough to run Shor's algorithm on large integers, the rapid advancements in quantum computing suggest that it may only be a matter of time before these classical systems become vulnerable [11].

Given this looming threat, the need to transition from classical cryptographic architectures to quantum-resistant alternatives is becoming increasingly urgent. Once quantum computers reach a certain level of maturity, traditional encryption methods could be rendered obsolete, exposing sensitive data to unprecedented risks. This impending vulnerability has spurred research into post-quantum cryptography/encryption schemes that remain secure even in the presence of quantum computing power.

Among the most promising candidates for post-quantum cryptography is lattice-based cryptography [22]. Unlike RSA or ECC, which rely on number-theoretic problems, lattice-based cryptographic schemes are based on the hardness of problems related to lattices in high-dimensional spaces. Currently, no quantum algorithms are known to efficiently solve these lattice problems, making them strong contenders for future cryptographic standards.

In this paper, I will explore the foundations of lattice-based cryptography, with a particular focus on the Learning With Errors (LWE) problem [22]. I will also examine the inherent hardness of lattice-based problems used in cryptography, their practical implementations, and their advantages in the context of post- quantum cryptography. Additionally, I will analyze Module-LWE Key Encapsulation Mechanism (ML-KEM) [21], a specific lattice-based scheme, which was officially selected as a cryptographic standard by NIST in August 2024. This analysis will break down the structure of ML- KEM and highlight its role in ensuring secure communication in a future where quantum computers are capable of breaking traditional cryptosystems

II. Basic principles

A. Mathematical foundations

In order to understand lattice-based cryptography, we must first define what a lattice is and describe its mathematical properties. A lattice Λ is a discrete subgroup of $\mathbb{R}^n$ formed by all integer linear combinations of $\boldsymbol{n}$ linearly independent vectors. More formally, let $B = \{b_1, b_2, \ldots, b_m\}$ be a set of linearly independent vectors in $\mathbb{R}^n$. The set of all integer linear combinations

$$L(B) = \left\{ \sum_i a_i b_i \mid a_i \in \mathbb{Z} \right\} \subset \mathbb{R}^n \tag{1}$$

is called a **lattice** in $\mathbb{R}^n$ with **basis** $\boldsymbol{B}$, **rank** $\boldsymbol{m}$ and **dimension** $\boldsymbol{n}$. When $m = n$ the lattice spans the entire space $\mathbb{R}^n$ and is called a **full-rank-lattice**. [17]

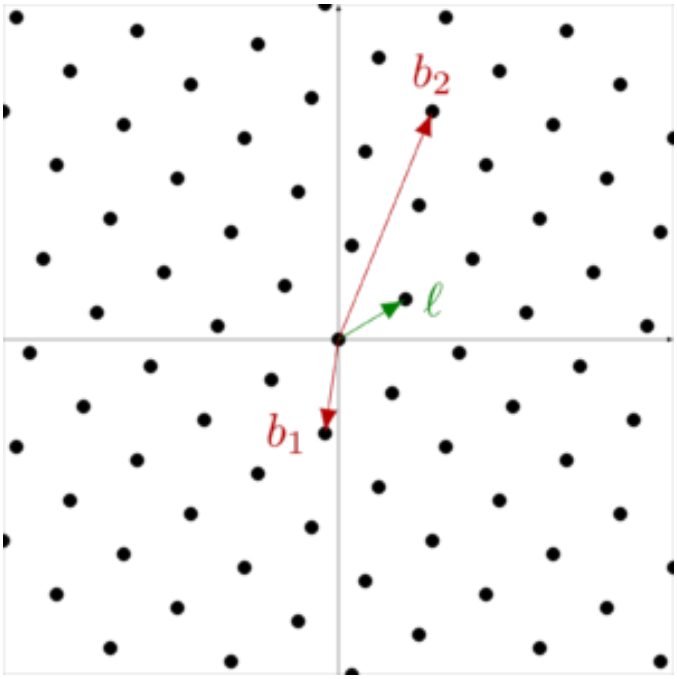


Fig. 1. Visualization of the Shortest Vector Problem which depicts a lattice in $\mathbb{R}^2$, with a basis $\{b_1, b_2\}$ and its shortest vector $\boldsymbol{l}$ [17]

B. Computational problems of lattices

It is important to note that a lattice can have multiple distinct basis and the basis used to represent the lattice can have a big impact on the hardness of different computational problems [17].

1) Basic problems: One such computational problem is the **shortest vector problem (SVP)** [20], which is the problem of finding the shortest non zero vector of a lattice given some basis $B \in \mathbb{R}^{n \times m}$ as seen in figure 1 which depicts a lattice in $\mathbb{R}^2$, with a basis $\{b_1, b_2\}$ and its shortest vector $\boldsymbol{l}$. Another fundamental problem is determining the closest lattice vector $\boldsymbol{l}$ to a given arbitrary vector in some lattice with basis $B \in \mathbb{R}^{n \times m}$ and norm $\|.\|$, called the **closest vector problem (CVP)** [19].

2) Learning with errors (LWE): Consider a basic linear equation

$$A \cdot s = b \mod q \tag{2}$$

where $A \in \mathbb{Z}^{n \times m}$, the solution $b \in \mathbb{Z}^n$ and the modulus $\boldsymbol{q}$ are known. Determining $s \in \mathbb{Z}^m$ can be easily done using the Gaussian algorithm [10]. Adding a small unknown error term $\boldsymbol{e}$:

$$A \cdot s + e = b \mod q \tag{3}$$

however makes it very difficult to determine $\boldsymbol{s}$. The small, unknown error term is called noise. One can interpret this problem as an instance of the **closest vector problem**. If you think of $A \cdot s$ as a lattice point and $\boldsymbol{b}$ as a perturbed version of this lattice point, the problem is essentially about finding the lattice vector that is closest to $\boldsymbol{b}$. More formally we define the lattice:

$$\Lambda = \{A \cdot x \mod q \mid x \in \mathbb{Z}_q^m\} \tag{4}$$

Given the lattice Λ and the target vector $\boldsymbol{b}$ we, have to find the lattice vector $v = A \cdot s$ where $\|b - v\|$ is minimized, since $\|e\| = \|b - A \cdot s\|$ [22].

3) Ring-LWE and Module-LWE: **Ring-LWE** [16] is an adaptation of LWE that operates over a polynomial ring $R_q := \mathbb{Z}_q[x]/f(x)$ for some polynomial $f(x)$ [14], which introduces algebraic structure to improve efficiency and uses polynomial multiplication in place of matrix multiplication. Specifically, in Ring-LWE, the standard LWE equations are translated into operations over polynomials, leveraging the ring structure to optimize computations. The key idea is that by representing elements as polynomials modulo f(x) and coefficients modulo q, one can perform convolutions (which are computationally intensive) more efficiently using algorithms like the Number Theoretic Transform (NTT) [4]. This results in faster key generation, encryption and decryption processes, as well as reduced key and ciphertext sizes compared to standard LWE.

Module-LWE [27] extends the concepts of both LWE and Ring-LWE, serving as a middle ground between them. In Module-LWE, the scalars in the LWE problem are replaced with ring elements from R_q, similar to Ring-LWE, but computations are performed over modules rather than rings. A module can be thought of as a higher-dimensional generalization of a vector space over a ring [6].

This approach retains some of the efficiency benefits of Ring-LWE while reducing the potential attack surface introduced by the ring's algebraic structure [27]. By operating over modules, Module-LWE allows for greater flexibility in parameter selection, enabling a balance between performance and security.

In Module-LWE, the problem is defined with the following components:

$$A \in (\mathbb{Z}_q[x]/f(x))^{n \times m}, \tag{5}$$

$$s \in (\mathbb{Z}_q[x]/f(x))^m \tag{6}$$

and

$$b, e \in (\mathbb{Z}_q[x]/f(x))^n \tag{7}$$

III. Hardness of computational problems of lattices

The most important metric to judge methods of encryption by is how secure and resistant to possible attacks they are [23]. One way to quantify and formalize this metric is to examine the hardness of the problems these algorithms are based on. Computational Complexity theory [12] defines hardness as the inherent difficulty of solving a problem characterized by the amount of resources required in relation to the input size. In the case of lattice based cryptography, we will look at the hardness of the fundamental computational problems defined in section II-B, as well as discussing some key features of these problems.

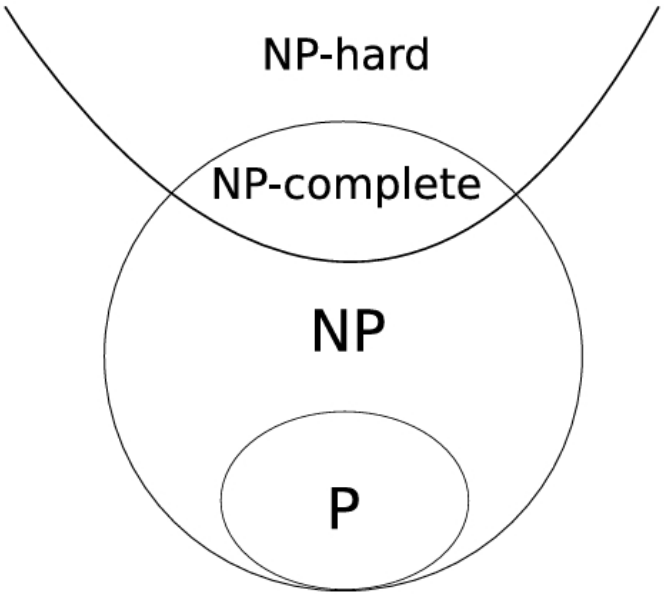


Fig. 2. Sets of NP, NP complete, NP hard and P [1]

A. Worst-case to average case reduction

One key feature of lattice based cryptography is the existence of reductions from hard worst-case lattice problems to the average hardness of certain problems [15]. We call $\boldsymbol{B}$ a reduction [9] of $\boldsymbol{A}$ if there exists a function $\boldsymbol{f}$ such that:

$$\forall a,\ a \in A \leftrightarrow f(a) \in B \tag{8}$$

This means that solving an instance of a problem requires the same algorithm as solving the hardest instance of the problem. This property provides strong security guarantees for cryptography algorithms based on lattices, as breaking them would imply solving problems that are hard in the worst case. In contrast, many classical cryptography algorithms rely on problems where there is no known worst-case to average-case reduction. Because of this, solving random instances of these problems does not imply you can efficiently solve the hardest instances of these problems. In practice, this makes key generation for lattice based cryptography easier and more secure than classical cryptography.

B. NP hardness of SVP and CVP

NP is the set of computational problems that can be solved in non-deterministic polynomial time. A problem is called NP-hard if it is at least as hard as every problem in NP. In particular if a problem is NP-hard, there is a reduction for every problem in NP to it, thus an NP-hard problem cannot be solved by a polynomial-time algorithm unless the computational hardness assumption $P \neq NP$ is false, with P being the set of computational problems that can be solved in deterministic polynomial time. A problem is NP-complete if it is both in NP and NP-hard [12]. The sets of P, NP and NP complete are visualized in figure (2). The shortest vector problem as well as the closest vector problem have been proven to be NP-hard [18]. Intuitively, this is due to the fact that the amount of possible points or solutions for a lattice problem increases exponentially with the dimension of the lattice. Furthermore, due to the discrete nature of lattices, gradient based methods cannot be used to solve lattice problems [17], which is a common technique to approximate complex search problems with non exponential effort (for example: the estimation of network weights in machine learning [29])

C. Reduction of LWE to CVP

Mathematically, the hardness of more complex lattice problems is proven by finding reductions to other problems which are know to be NP-hard. LWE has been shown to be at least as hard as CVP by Regev's reduction [22]. However, the proof and the underlying mathematics would be too much for the format of this paper. A more in depth dive into the topic can be found in [13].

D. Hardness of Ring-LWE vs LWE and Module-LWE

While Ring-LWE offers significant efficiency gains, the additional algebraic structure introduces potential avenues for attack. The structured nature of the ring could, in theory, be exploited by specialized algorithms aiming to solve the underlying hard problems more efficiently than in the unstructured case of standard LWE [5]. However, as of today, no specific attacks have been successful in undermining the security of Ring-LWE when appropriate parameters are chosen. The problem remains considered hard, and Ring-LWE-based schemes are widely regarded as secure, while Module-LWE is considered to be as hard as LWE [27].

E. Hardness of lattice based cryptography in the context of PQC

Currently, there are no known algorithms,neither classical nor quantum, that can efficiently solve problems like Learning With Errors (LWE). The two quantum algorithms that pose the greatest threat to classical cryptography are Shor's algorithm [26] and Grover's algorithm [28]. However, neither of these algorithms is effective against lattice-based cryptography.

Grover's algorithm, which offers a quadratic speedup for brute-force searches, still isn't powerful enough to undermine the NP-hardness of lattice problems, as discussed in section III-B.

Shor's algorithm, on the other hand, is specifically designed to exploit the periodicity in problems like prime factorization and discrete logarithms — features that are not relevant to lattice problems [26]. In addition to their resilience against quantum attacks, lattice problems offer other practical advantages that make them particularly well-suited for post-quantum cryptography. For instance, as detailed in section III-A, lattice-based schemes benefit from worst-case to average-case reductions, providing strong security guarantees. Furthermore, lattice problems can be efficiently implemented using classical computers, making them practical for both encryption and decryption in today's systems. These factors — quantum

resistance, worst-case security guarantees, and practical implementation — position lattice-based cryptography as a critical component for the future of post-quantum cryptography.

F. Possible attacks on lattice based cryptography using Quantum ML

While lattices based cryptography is resistant to both classical and quantum attacks there are methods using quantum machine learning [7] to approximate lattice problems. However, as of today none of these are accurate or efficient enough to pose a significant threat to lattice based cryptography and analyzing these methods in detail would be too much for the scope of this paper. More detail can be found in [2].

IV. A CLOSER LOOK AT ML-KEM

In August of 2024, the US based National Institute of Standards and Technology [21] has released its first three finalized post-quantum encryption standards, two of which are based on algorithms using Module LWE. More specifically the Federal Information Processing Standard (FIPS) 203, intended as the primary standard for general encryption, is based on the CRYSTALS-Kyber algorithm [8], which has been renamed ML-KEM, short for Module-Lattice-Based Key-Encapsulation Mechanism. In order to understand how exactly lattice problems are applied to cryptography, we will now take a closer look at the ML-KEM algorithm and analyze how LWE and its variations are used to encrypt and decrypt messages.

A. Basic cryptography concepts

Understanding ML-KEM requires a basic understanding of the difference between symmetric and asymmetric encryption [24]. In **symmetric encryption**, both parties (the sender and receiver) share a single secret key, which is used for both encrypting and decrypting messages. This method is simple and efficient but requires secure key exchange, as both parties need access to the same key. In **asymmetric encryption**, two different keys are used: a public key and a private key. The public key is available to everyone and is used to encrypt messages, while the private key, known only to the recipient, is used to decrypt them. This method eliminates the need for sharing a secret key in advance. A widely used example of asymmetric encryption is RSA.

Today, most communication systems use a combination of these methods in what is known as **hybrid encryption** [25]. Hybrid encryption leverages the speed of symmetric encryption for data transmission and the security of asymmetric encryption for securely exchanging the symmetric key. One common hybrid encryption approach is the Key Encapsulation Mechanism (KEM), which securely transfers a symmetric key using asymmetric encryption.

B. Key-Encapsulation-Mechanism (KEM)

A Key-Encapsulation-Mechanism (KEM) [21] is a **hybrid** cryptographic scheme used to establish a secret key between two communicating parties and consists of three underlying algorithms. The first of which is **KeyGen**. As seen in figure 3, it generates a public encapsulation key, which Bob uses to run **Encaps**. This generates Bob's copy of the shared secret key, along with an associated ciphertext. The ciphertext is then sent to Alice. Alice, using her private decapsulation key (also generated by KeyGen), runs the **Decaps** operation to obtain her version of the shared secret key.

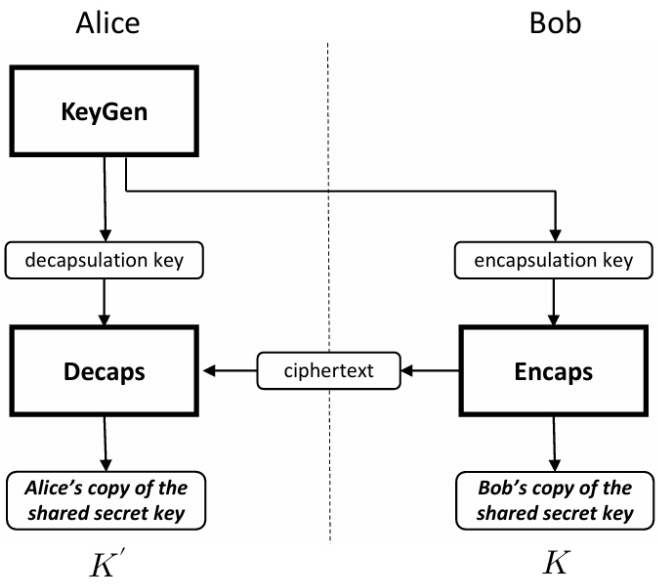


Fig. 3. A simple view of key establishment using a KEM [21]

C. Structure of ML-KEM

The structure of ML-KEM as defined in the NIST standard [21] follows that of a classic Key-Encapsulation-Mechanism as detailed in section IV-B. In the context of this paper, we will focus on the KeygGen subroutine as well as the encyrption and decryption of the shared symmetric key, while not going into detail on the creation of the symmetric key and specific subroutines used to sample random vectors or transform certain parameters to allow for more efficient calculations. More detail on these aspects can be found in [21].

D. KeygGen

The KeyGen algorithm as described in the NIST standard, requires no input. Its output consists of an encapsulation key ek and a decapsulation key dk. Firstly two variables $\boldsymbol{d}$ and $\boldsymbol{z}$ are instantiated as 32 random bytes. These are then used as input to run the subroutine **ML-KEM.KeyGen-internal(d, z)**.

1) KeyGen-internal: This subroutine immediatley runs yet another subroutine called **K-PKE.KeyGen(d)** to which it passes its parameter $\boldsymbol{d}$. This subroutine returns an encryption key which is used as the encaps key for the KEM and a decryption key which is transformed into the decaps key by:

$$\text{dk} \leftarrow (\text{dk}_{\text{PKE}} \parallel \text{ek} \parallel H(\text{ek}) \parallel z) \tag{9}$$

with $\boldsymbol{dk}$ being the final decaps key, dk_{PKE} being the decryption key generated by K-PKE.KeyGen(d) and **H(ek)** the hash

Algorithm 13 K-PKE.KeyGen(d)

Uses randomness to generate an encryption key and a corresponding decryption key.

Input: randomness $d \in \mathbb{B}^{32}$.
Output: encryption key $\text{ek}_{\text{PKE}} \in \mathbb{B}^{384k+32}$.
Output: decryption key $\text{dk}_{\text{PKE}} \in \mathbb{B}^{384k}$.

1: $(\rho, \sigma) \leftarrow G(d\|k)$ ▷ expand 32+1 bytes to two pseudorandom 32-byte seeds[1]
2: $N \leftarrow 0$
3: **for** ($i \leftarrow 0; i < k; i$++) ▷ generate matrix $\hat{\mathbf{A}} \in (\mathbb{Z}_q^{256})^{k \times k}$
4: **for** ($j \leftarrow 0; j < k; j$++)
5: $\hat{\mathbf{A}}[i,j] \leftarrow \text{SampleNTT}(\rho\|j\|i)$ ▷ j and i are bytes 33 and 34 of the input
6: **end for**
7: **end for**
8: **for** ($i \leftarrow 0; i < k; i$++) ▷ generate $\mathbf{s} \in (\mathbb{Z}_q^{256})^k$
9: $\mathbf{s}[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N))$ ▷ $\mathbf{s}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
10: $N \leftarrow N+1$
11: **end for**
12: **for** ($i \leftarrow 0; i < k; i$++) ▷ generate $\mathbf{e} \in (\mathbb{Z}_q^{256})^k$
13: $\mathbf{e}[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N))$ ▷ $\mathbf{e}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
14: $N \leftarrow N+1$
15: **end for**
16: $\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s})$ ▷ run NTT k times (once for each coordinate of $\mathbf{s}$)
17: $\hat{\mathbf{e}} \leftarrow \text{NTT}(\mathbf{e})$ ▷ run NTT k times
18: $\hat{\mathbf{t}} \leftarrow \hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}}$ ▷ noisy linear system in NTT domain
19: $\text{ek}_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{\mathbf{t}})\|\rho$ ▷ run ByteEncode_{12} k times, then append $\hat{\mathbf{A}}$-seed
20: $\text{dk}_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{\mathbf{s}})$ ▷ run ByteEncode_{12} k times
21: **return** $(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}})$

Fig. 4. Pseudocode for K-PKE.KeyGen as detailed in NIST publication [21]

of the encryption key **ek** using a cryptographic hash function $\boldsymbol{H}$, while the operator $\|$ denotes the concatenation of byte strings or bit strings.

2) K-PKE.KeyGen: When considering the algorithm 4 it becomes clear that it follows the structure of Module-LWE introduced in section II-B. It starts by using a pseudorandom Generator $\boldsymbol{G}$ to expand the input d concatenated with a security parameter $\boldsymbol{k}$ into two 32-byte pseudorandom seeds ρ and σ (line 1). ρ is then used to generate the Matrix $\mathbf{A} \in \mathbb{Z}_{256}^{k \times k}$ (lines 3-7), while σ is used to generate the secret vector $\mathbf{s} \in \mathbb{Z}_{256}^{k}$ (lines 8-11) and the error $e \in \left(\mathbb{Z}_q^{256}\right)^k$ (lines 12-15). The transforms $\hat{s} \leftarrow \text{NTT}(s)$ and $\hat{e} \leftarrow \text{NTT}(e)$ (lines 16, 17) are done to allow efficient polynomal multiplication and realize the use of Module-LWE, instead of normal LWE. Now we have a noisy linear equation

$$t = A * s + e \tag{10}$$

with A, s, e and t having the exact same definitions as in II-B (line 18). The subroutine then returns the noisy linear equation as the public encryption key and the secret vector $\boldsymbol{s}$ as the private decryption key after encoding them into a byte array for $1 \leq d \leq 12$ (lines 19-21).

E. Regevs Encryption Scheme

Regev's encryption scheme [3] provides a method to apply the Learning with Errors (LWE) problem to encryption, forming a foundation for secure public-key cryptosystems. In the context of the Module Learning with Errors Key Encapsulation Mechanism (ML-KEM), we focus on the public-key encryption version of this scheme, adhering to the principles of asymmetric encryption explained in section IV-A.

To encrypt a message $\boldsymbol{m}$, given an encryption key $\boldsymbol{ek}$ and randomness $\boldsymbol{r}$, Regev's scheme performs several steps. First, it generates a random vector $\boldsymbol{y}$ and samples small error vectors e_1 and e_2 to introduce noise and ensure security. The encryption process computes two components, $\boldsymbol{u}$ and $\boldsymbol{v}$, which together form the ciphertext.

The first component, $\boldsymbol{u}$, is calculated by multiplying the transpose of the public matrix $\boldsymbol{A}$ with the random vector $\boldsymbol{y}$, adding the error vector e_1 and then reducing the result modulo q:

$$u = A^\top y + e_1 \mod q \tag{11}$$

The second component, $\boldsymbol{v}$, is computed by multiplying the transpose of $\boldsymbol{t}$ with $\boldsymbol{y}$ adding the error term e_2 and incorporating the encoded message μ, then reducing modulo q:

$$v = t^\top y + e_2 + \mu \mod q \tag{12}$$

Here, t is derived from the public key and is defined as $t = A \cdot s + e \mod q$, where $\boldsymbol{s}$ is the secret key and $\boldsymbol{e}$ is another small error vector. The encoded message μ represents the plaintext $\boldsymbol{M}$. The use of a randomness with $\boldsymbol{y}$ ensures that each encryption is unique, enhancing security by preventing attackers from detecting patterns in the ciphertext. The ciphertext is then the pair (u, v), which encapsulates the encrypted message along with the necessary components to allow decryption by the intended recipient.

For decryption, the owner of the secret key $\boldsymbol{s}$ computes a value ν by subtracting the inner product of $\boldsymbol{u}$ and $\boldsymbol{s}$ from $\boldsymbol{v}$, modulo q:

$$\nu = v - u \cdot s \mod q \tag{13}$$

Because the error terms e_1, e_2 and $\boldsymbol{e}$ are small relative to $\boldsymbol{q}$, this computation effectively cancels out the contributions of $\boldsymbol{y}$ and the errors, leaving a result that is close to the encoded message μ. The recipient can then recover μ and, consequently, the original message $\boldsymbol{m}$ by mapping $\boldsymbol{v}$ back to the plaintext space, accounting for any potential rounding or errors introduced during encryption. This scheme demonstrates how the LWE problem can be leveraged for secure encryption. The incorporation of small error terms and randomness ensures that without knowledge of the secret key $\boldsymbol{s}$, it is computationally infeasible for an adversary to recover the original message from the ciphertext.

F. Encaps

The Encaps algorithm requires an encapsulation key ek as an input. In the context of the KEM this is the public encryption key generated in KeyGen. Afterwards the secret key to be transmitted to Alice is derived and encrypted using the subroutine **K-PKE.Encrypt**.

1) K-PKE.Encrypt: This subroutine seen in figure 5 takes as input an encryption key ek, a message m and a randomness r. It outputs a ciphertext c.

Initially, $\hat{t}$ is decoded from the public key, corresponding to equation 10 where $\boldsymbol{A}$ is the public matrix, $\boldsymbol{s}$ is the secret key, and $\boldsymbol{e}$ is the error vector (line 2). The seed ρ is then

Algorithm 14 K-PKE.Encrypt($\mathsf{ek}_{\mathsf{PKE}}, m, r$)

Uses the encryption key to encrypt a plaintext message using the randomness r.

Input: encryption key $\mathsf{ek}_{\mathsf{PKE}} \in \mathbb{B}^{384k+32}$.
Input: message $m \in \mathbb{B}^{32}$.
Input: randomness $r \in \mathbb{B}^{32}$.
Output: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.

1: $N \leftarrow 0$
2: $\hat{\mathbf{t}} \leftarrow \mathsf{ByteDecode}_{12}(\mathsf{ek}_{\mathsf{PKE}}[0:384k])$ ▷ run $\mathsf{ByteDecode}_{12}$ k times to decode $\hat{\mathbf{t}} \in (\mathbb{Z}_q^{256})^k$
3: $\rho \leftarrow \mathsf{ek}_{\mathsf{PKE}}[384k:384k+32]$ ▷ extract 32-byte seed from $\mathsf{ek}_{\mathsf{PKE}}$
4: **for** $(i \leftarrow 0; i < k; i$++$)$ ▷ re-generate matrix $\hat{\mathbf{A}} \in (\mathbb{Z}_q^{256})^{k \times k}$ sampled in Alg. 13
5: **for** $(j \leftarrow 0; j < k; j$++$)$
6: $\hat{\mathbf{A}}[i,j] \leftarrow \mathsf{SampleNTT}(\rho \| j \| i)$ ▷ j and i are bytes 33 and 34 of the input
7: **end for**
8: **end for**
9: **for** $(i \leftarrow 0; i < k; i$++$)$ ▷ generate $\mathbf{y} \in (\mathbb{Z}_q^{256})^k$
10: $\mathbf{y}[i] \leftarrow \mathsf{SamplePolyCBD}_{\eta_1}(\mathsf{PRF}_{\eta_1}(r, N))$ ▷ $\mathbf{y}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
11: $N \leftarrow N+1$
12: **end for**
13: **for** $(i \leftarrow 0; i < k; i$++$)$ ▷ generate $\mathbf{e_1} \in (\mathbb{Z}_q^{256})^k$
14: $\mathbf{e_1}[i] \leftarrow \mathsf{SamplePolyCBD}_{\eta_2}(\mathsf{PRF}_{\eta_2}(r, N))$ ▷ $\mathbf{e_1}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
15: $N \leftarrow N+1$
16: **end for**
17: $e_2 \leftarrow \mathsf{SamplePolyCBD}_{\eta_2}(\mathsf{PRF}_{\eta_2}(r, N))$ ▷ sample $e_2 \in \mathbb{Z}_q^{256}$ from CBD
18: $\hat{\mathbf{y}} \leftarrow \mathsf{NTT}(\mathbf{y})$ ▷ run NTT k times
19: $\mathbf{u} \leftarrow \mathsf{NTT}^{-1}(\hat{\mathbf{A}}^\top \circ \hat{\mathbf{y}}) + \mathbf{e_1}$ ▷ run NTT^{-1} k times
20: $\mu \leftarrow \mathsf{Decompress}_1(\mathsf{ByteDecode}_1(m))$
21: $v \leftarrow \mathsf{NTT}^{-1}(\hat{\mathbf{t}}^\top \circ \hat{\mathbf{y}}) + e_2 + \mu$ ▷ encode plaintext m into polynomial v
22: $c_1 \leftarrow \mathsf{ByteEncode}_{d_u}(\mathsf{Compress}_{d_u}(\mathbf{u}))$ ▷ run $\mathsf{ByteEncode}_{d_u}$ and $\mathsf{Compress}_{d_u}$ k times
23: $c_2 \leftarrow \mathsf{ByteEncode}_{d_v}(\mathsf{Compress}_{d_v}(v))$
24: **return** $c \leftarrow (c_1 \| c_2)$

Fig. 5. Pseudocode for K-PKE.Encrypt as detailed in NIST publication [21]

extracted from the public key and used to regenerate the matrix A $\in Z_{256}^{k \times k}$ (lines 3-8) .

Next, a random vector $y \in \left(\mathbb{Z}_q^{256}\right)^k$ is sampled using the randomness $\boldsymbol{r}$ (lines 9-12). This ensures that each encryption is unique, preventing potential attackers from inferring information about the message through patterns in the ciphertext. Two error vectors, e_1 and e_2 are also sampled using $\boldsymbol{r}$ to introduce additional noise into the ciphertext (lines 13-17).

The first part of the ciphertext, $\boldsymbol{u}$, is computed by multiplying the transpose of $\boldsymbol{A}$ with $\boldsymbol{y}$ and adding the noise $\boldsymbol{e_1}$ (line 19):

$$u = A^\top y + e_1 \mod q \tag{14}$$

The second part of the ciphertext, $\boldsymbol{v}$, is calculated by multiplying the transpose of $\boldsymbol{t}$ with $\boldsymbol{y}$, adding the noise $\boldsymbol{e_2}$ and the message polynomal $\boldsymbol{\mu}$ (line 21):

$$v = \hat{t}^\top y + e_2 + \mu \mod q \tag{15}$$

Both $\boldsymbol{u}$ and $\boldsymbol{v}$ are then compressed and encoded into byte strings for transmission. They are concatenated to form the final ciphertext $c = (u, v)$ (lines 22-24).

In summary, this algorithm follows Regev's encryption scheme for the Learning with Errors (LWE) problem, while incorporating Module-LWE and the Number Theoretic Transform (NTT) to enhance efficiency. The use of Module-LWE allows for operations on polynomials rather than individual bits, significantly improving performance. Without Module-LWE, each bit would need to be computed separately, leading to increased computational overhead.

Algorithm 15 K-PKE.Decrypt($\mathsf{dk}_{\mathsf{PKE}}, c$)

Uses the decryption key to decrypt a ciphertext.

Input: decryption key $\mathsf{dk}_{\mathsf{PKE}} \in \mathbb{B}^{384k}$.
Input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.
Output: message $m \in \mathbb{B}^{32}$.

1: $c_1 \leftarrow c[0:32d_u k]$
2: $c_2 \leftarrow c[32d_u k : 32(d_u k + d_v)]$
3: $\mathbf{u}' \leftarrow \mathsf{Decompress}_{d_u}(\mathsf{ByteDecode}_{d_u}(c_1))$ ▷ run $\mathsf{Decompress}_{d_u}$ and $\mathsf{ByteDecode}_{d_u}$ k times
4: $v' \leftarrow \mathsf{Decompress}_{d_v}(\mathsf{ByteDecode}_{d_v}(c_2))$
5: $\hat{\mathbf{s}} \leftarrow \mathsf{ByteDecode}_{12}(\mathsf{dk}_{\mathsf{PKE}})$ ▷ run $\mathsf{ByteDecode}_{12}$ k times
6: $w \leftarrow v' - \mathsf{NTT}^{-1}(\hat{\mathbf{s}}^\top \circ \mathsf{NTT}(\mathbf{u}'))$ ▷ run NTT k times; run NTT^{-1} once
7: $m \leftarrow \mathsf{ByteEncode}_1(\mathsf{Compress}_1(w))$ ▷ decode plaintext m from polynomial v
8: **return** m

Fig. 6. Pseudocode for K-PKE.Decrypt as detailed in NIST publication [21]

G. Decaps

The Decaps algorithm has a decapsulation key dk as well as a cyphertext c as inputs and returns a shared secret key y_k. This is realized in an internal decapsulation algorithm **ML-KEM.Decaps-internal(dk, c)**.

1) Decaps-internal: This subroutine extracts the public encryption as well as the secret decryption key from the decapsulation key. It then runs the subroutine **K-PKE.Decrypt** 6 to decrypt the ciphertext. Afterwards the message is re-encrypted and the algorithm checks if the two generated ciphertexts are the same, to ensure encryption and decryption was done correctly.

2) K-PKE.Decrypt: As seen in figure 6, first c_1 and c_2 are extracted from the ciphertext, corresponding to the two parts of the ciphertext detailed in the section IV-F1 (lines 1, 2). These are then converted from bytes to their numerical representation and expanded to their original size (lines 3, 4). Next the secret key $\boldsymbol{s}$ is decoded from its byte representation and converted to its numerical representation as well (line 5). Afterwards the message is decoded by essentially calculating:

$$\nu = c_2 - c_1 \cdot s \mod q \tag{16}$$

which is then compressed, converted back to byte form and returned (lines 5-7). Once again the decryption follows Regevs encryption scheme.

V. Conclusion

In this paper, we have explored the foundations of lattice-based cryptography, with a particular emphasis on the Learning with Errors (LWE) problem. We examined the computational hardness of lattice problems within the context of post-quantum cryptography and outlined key features that contribute to their security and practicality. By analyzing the NIST-standard algorithm ML-KEM, we demonstrated how lattice problems are effectively applied in cryptographic schemes to provide quantum-resistant security.

The rapid advancements in quantum computing underscore the urgent need to adopt cryptographic methods that are resilient against quantum attacks, especially for safeguarding critical infrastructure. Lattice-based cryptography emerges as a promising candidate due to its strong security guarantees

and efficiency. Its reliance on the hardness of lattice problems, which remain intractable even for quantum computers, makes it a vital component in the development of future-proof cryptographic systems.

Future research directions include investigating reductions of complex lattice problems to simpler ones to identify potential vulnerabilities and enhance security. Additionally, exploring the application of quantum machine learning techniques to approximate lattice problems presents both opportunities and challenges that warrant further study. Understanding these aspects is crucial for anticipating and mitigating new types of attacks that may arise in the quantum computing era.

In conclusion, lattice-based cryptography offers a practical and robust alternative to classical encryption methods, providing the necessary quantum resistance as we move towards an increasingly quantum-capable future. Its significance is expected to grow, making it an essential focus for both theoretical research and practical implementation in securing communications against emerging threats.

REFERENCES

[1] Scott Aaronson. P=\limitsˆ? np. In *Open problems in mathematics*, pages 1–122. Springer, 2016.

[2] Martin R Albrecht, Miloš Prokop, Yixin Shen, and Petros Wallden. Variational quantum solutions to the shortest vector problem. *Quantum*, 7:933, 2023.

[3] Prabhanjan Ananth, Alexander Poremba, and Vinod Vaikuntanathan. Revocable cryptography from learning with errors. In Guy Rothblum and Hoeteck Wee, editors, *Theory of Cryptography*, pages 93–122, Cham, 2023. Springer Nature Switzerland.

[4] Hanho Lee1 Ardianto Satriawan1, Rella Mareta1. A complete beginner guide to the number theoretic transform (ntt). 2024. URL: https://eprint.iacr.org/2024/585.pdf.

[5] David Balbás. The hardness of lwe and ring-lwe: A survey. *Cryptology ePrint Archive*, 2021.

[6] J. A. Beachy. *Introductory Lectures on Rings and Modules*. Cambridge University Press, 1999.

[7] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.

[8] Joppe Bos, Leo Ducas, Eike Kiltz, T Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. Crystals - kyber: A cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroSP)*, pages 353–367, 2018. https://doi.org/10.1109/EuroSP.2018.00032 doi:10.1109/EuroSP.2018.00032.

[9] Peter Bürgisser. *Completeness and Reduction in Algebraic Complexity Theory*. Springer, 2000.

[10] Gerd Fischer. *Lineare Algebra: Eine Einführung für Studienanfänger*. Vieweg-Verlag, 2002.

[11] Sukhpal Singh Gill, Adarsh Kumar, Harvinder Singh, Manmeet Singh, Kamalpreet Kaur, Muhammad Usman, and Rajkumar Buyya. Quantum computing: A taxonomy, systematic review and future directions. *Software: Practice and Experience*, 52(1):66–114, 2022.

[12] Oded Goldreich. *P, NP, and NP-Completeness: The basics of computational complexity*. Cambridge University Press, 2010. URL: https://doi.org/10.1017/CBO9780511761355.

[13] Yasuhito Kawano. A reduction from an lwe problem to maximum independent set problems. *Scientific Reports*, 13(1):7130, 2023.

[14] Serge Lang. *Algebra. 3. Auflage, Graduate Texts in Mathematics*. Springer, 2005.

[15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[16] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29*, pages 1–23. Springer, 2010.

[17] Magdalena Bertram Maximilian Richter, Jasper Seidensticker. A (somewhat) gentle introduction to lattice-based post-quantum cryptography. 2023. URL: https://www.cybersecurity.blog.aisec.fraunhofer.de/en/a-somewhat-gentle-introduction-to-lattice-based-post-quantum-cryptography.

[18] Daniele Micciancio. *On the hardness of the shortest vector problem*. PhD thesis, Massachusetts Institute of Technology, 1998.

[19] Daniele Micciancio. Closest vector problem. In Henk C. A. van Tilborg, editor, *Encyclopedia of Cryptography and Security*, pages 79–80. Springer US, Boston, MA, 2005. https://doi.org/10.1007/0-387-23483-$7_66doi : 10.1007/0 - 387 - 23483 - 7_66$.

[20] Daniele Micciancio. Shortest vector problem. In Henk C. A. van Tilborg and Sushil Jajodia, editors, *Encyclopedia of Cryptography and Security*, pages 1196–1197. Springer US, Boston, MA, 2011. https://doi.org/10.1007/978-1-4419-5906-$5_434doi : 10.1007/978 - 1 - 4419 - 5906 - 5_434$.

[21] Gina M Raimondo and Laurie E Locascio. Module-lattice-based key-encapsulation mechanism standard. *National Institute of Standards and Technology, Gaithersburg*, 2024. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf.

[22] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.

[23] Edgar C Reinke. Classical cryptography. *The Classical Journal*, 58(3):113–121, 1962.

[24] Priasnyomo Prima Santoso, Elkin Rilvani, Ahmad Budi Trisnawan, Krisna Adiyarta, Darmawan Napitupulu, Tata Sutabri, and Robbi Rahim. Systematic literature review: comparison study of symmetric key and asymmetric key algorithm. In *IOP Conference Series: Materials Science and Engineering*, volume 420, page 012111. IOP Publishing, 2018.

[25] Bruce Schneier. *Applied Cryptography. Protocols, Algorithms, and Source Code in C, Second Edition*. Springer, 1996.

[26] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997. URL: http://dx.doi.org/10.1137/S0097539795293172, https://doi.org/10.1137/s0097539795293172 doi:10.1137/s0097539795293172.

[27] Yang Wang and Mingqiang Wang. Module-lwe versus ring-lwe, revisited. *Cryptology ePrint Archive*, 2019.

[28] Hiu Yung Wong. Grover's algorithm: I. In *Introduction to Quantum Computing: From a Layperson to a Programmer in 30 Steps*, pages 221–230. Springer International Publishing, Cham, 2024. https://doi.org/10.1007/978-3-031-36985-$8_23doi : 10.1007/978 - 3 - 031 - 36985 - 8_23$.

[29] Zhi-Hua Zhou. *Machine learning*. Springer nature, 2021.